

Ex01 – Programmatic Bluetooth (Resumo Explicativo e Simplificado)

1. O que é o Bluetooth

O Bluetooth é uma tecnologia de comunicação sem fios usada para ligar dispositivos próximos, como telemóveis, auscultadores, computadores e sensores. A sua principal função é substituir cabos, permitindo a troca de dados de forma simples e automática.

Funciona na frequência de 2.4 GHz e tem um alcance curto (normalmente até 10 metros). Divide-se em diferentes camadas: o hardware (adaptador Bluetooth), o software (stack BlueZ no Linux) e as aplicações que usam serviços específicos (como áudio, ficheiros ou chat).

2. BlueZ, PyBluez e PyDBus

BlueZ é o sistema Bluetooth do Linux. Ele controla o adaptador e gere as ligações. **PyBluez** é uma biblioteca Python que permite programar Bluetooth diretamente (fazer scan, encontrar serviços, criar sockets, etc.). **PyDBus** é usada para funções mais avançadas, como emparelhar e conectar dispositivos, porque comunica com o sistema através do **D-Bus**.

3. Scanning (descoberta de dispositivos)

Antes de ligar a um dispositivo, é preciso encontrá-lo. O scan procura dispositivos Bluetooth próximos e mostra o nome e o endereço MAC de cada um.

- **Comando:** `bluetooth.discover_devices(lookup_names=True, duration=10)`
- O parâmetro *duration* define o tempo da pesquisa.
- Se o dispositivo estiver invisível, não aparece na lista.

4. Obter o nome e serviços do dispositivo

Depois de saber o MAC Address, podes confirmar o nome e ver os serviços disponíveis.

- **Nome:** `bluetooth.lookup_name(mac_address, timeout=10)`
- **Serviços:** `bluetooth.find_service(address=mac_address)`
- Os serviços mostram o tipo de ligação que o dispositivo suporta (ex: áudio, ficheiros, série).

5. Pareamento (Pairing)

Emparelhar é o processo de criar confiança entre dois dispositivos. É como 'autorizar' a ligação, trocando uma chave de autenticação. O Bluetooth guarda esta chave para futuras ligações.

- Feito com PyDBus, usando `device.Pair()`
- O `bluetoothctl` deve estar aberto como agente para confirmar o PIN.
- Depois de emparelhados, os dispositivos reconhecem-se automaticamente.

6. Conexão (Connect)

Depois do emparelhamento, é possível ligar de facto ao dispositivo para usar os seus serviços. O comando **device.Connect()** ativa os serviços disponíveis, como áudio ou troca de dados.

- A ligação só é possível se ambos os dispositivos suportarem o mesmo serviço.

- Se falhar, pode ser necessário repetir o pareamento.

7. Chat via Bluetooth (RFCOMM)

O RFCOMM é como uma porta série virtual sobre Bluetooth. Permite enviar e receber mensagens de texto, sem necessidade de cabos. O exercício cria um chat simples entre dois computadores.

- **Servidor:** cria socket RFCOMM, faz bind e espera ligações.
- **Cliente:** descobre dispositivos, escolhe um e conecta-se ao porto (geralmente 1).
- Ambos enviam e recebem mensagens dentro de um loop.
- O servidor precisa estar 'discoverable' para o cliente o encontrar.

8. Dicas e boas práticas

- Desativa o modo 'discoverable' quando não precisares dele.
- Testa sempre o scan com diferentes durações.
- Trata exceções no código (BluetoothError, Timeout).
- Atualiza o BlueZ para evitar falhas conhecidas (como BlueBorne).

9. Perguntas possíveis (e respostas curtas)

- ■ O que faz o comando `bluetooth.discover_devices()`?
→■ Procura dispositivos Bluetooth próximos.
- ■ O que é o RFCOMM?
→■ Um protocolo que simula uma porta série via Bluetooth.
- ■ Para que serve o `bluetoothctl` durante o pareamento?
→■ Atua como agente que aceita o PIN.
- ■ O que acontece se o dispositivo não estiver 'discoverable'?
→■ O scan não o encontra.
- ■ Qual a função do PyDBus?
→■ Permite aceder à API Bluetooth via D-Bus (pareamento e conexão).
- ■ Por que o Bluetooth usa a frequência 2.4 GHz?
→■ É uma banda ISM livre, usada também por Wi-Fi.
- ■ O que acontece se um dispositivo mudar de PIN?
→■ O emparelhamento antigo deixa de ser válido.